

NOI2026 模拟赛 Day6 题目解析

云浅

2026-7-14

Peking University

1. 树的重心	2
2. 取石子游戏	7
3. 排列计数	16

1. 树的重心

1. 树的重心

有一棵 n 个结点的树，每个点有点权 a_i 。

现在云浅会进行 q 次询问，第 i 次询问给出树上的 k_i 条边。如果断掉这 k_i 条边，那么树会分裂成 $k_i + 1$ 个连通块。你需要对每个连通块 P 找到一个点 u ，使得

$$\sum_{v \in P} \text{dist}(u, v) \times a_v$$

最小。这里 $\text{dist}(u, v)$ 指的是树上 u, v 两点之间最短路径上的边数。如果有多个满足条件的点，取编号最小的那个。

注意：询问之间互相独立，也就是说并没有真的断掉这些边。

数据范围： $1 \leq n, q, \sum k_i \leq 2 \times 10^5, 1 \leq a_i \leq 10^4$

1.1. 算法一

可以每次 $O(n)$ 求出重心，时间复杂度 $O(nq)$ 。

1.2. 算法二

对于树是一条链的情况，相当于对每个区间找带权中点，二分即可。

时间复杂度 $O(n + (q + \sum k_i) \log n)$

1.3. 算法三

考虑怎么算带权重心：按照 DFS 序遍历，第 i 个点重复写 a_i 次，那么重心在根到第 $\lceil \frac{s}{2} \rceil$ 个点的路径上。其中 $s = \sum a_i$ 。证明考虑每次如果 size 最大的子树的大小严格超过了全局的一半我们就往那边走，发现既然超过了全局一半那么它在 DFS 序上占据的区间长度一定 $\geq \lceil \frac{s}{2} \rceil + 1$ ，于是第 $\lceil \frac{s}{2} \rceil$ 个点一定在这个区间内。于是我们第一步（如果能走）就走对了，依此类推，每一步都能走对。

那么找到这个点之后，在这条路径上之后就好做了。我们二分/倍增，每次 check 当前点有没有走过头，也就是找路径上最靠下的一个点满足它的子树 $\text{size} \geq \lceil \frac{s}{2} \rceil + 1$ 。

另一方面，找这个点也是好找的，同样是在 DFS 序上二分即可。这里我们算一个子树的 size 的时候可以直接暴力扫描它下方的所有儿子，并减掉对应的 size，这样复杂度是 $O(n + (q + \sum k_i) \log n)$ 的。

2. 取石子游戏

2. 取石子游戏

Alice 和 Bob 在玩一个取石子游戏。有一堆石子，初始数量为 10^{100} 。

每次操作，玩家可以从石子堆中取走 $[1, m]$ 范围内的任意整数个石子 x 。

但是，**不能**取走对手上一回合刚刚取走的那个数量 x （第一回合没有限制，Alice 可以取 $[1, m]$ 中任意数量）。

谁取走第 n 个石子（即取完后总取走数量至少为 n ），谁就获胜。

请判断 Alice 是否有必胜策略。

数据范围： $1 \leq n \leq 10^6, 2 \leq m \leq 10^6$ ，有 $t \leq 3 \times 10^5$ 组询问。

2.1. 算法一

如果 m 是偶数，那么先手必败当且仅当 $m + 1$ 是 n 的约数。

证明：如果初始 $(m + 1) \mid n$ ，那么每次如果先手取 x 个，后手只需要取 $m + 1 - x$ 个（注意由于 m 是偶数， $m + 1 - x \neq x$ ）即可回到 $(m + 1) \mid n$ 的状态，最后由后手到达 0。反之，先手总可以进行一步操作使得 $(m + 1) \nmid n$ 。

那么这种情况可以 $O(1)$ 判断。可以获得 16 分。

从这里开始，我们的讨论都假设 m 为奇数。

2.2. 算法二

可以设计 $\text{dp}(i, j)$ 表示希望取第 i 个石子，上一次取了 j 个的情况下，先手是否必胜。

转移形如：如果存在 $1 \leq x \leq m, x \neq j$ 使得 $\text{dp}(i - x, x) = \text{false}$ ，则 $\text{dp}(i, j) = \text{true}$ ，否则 $\text{dp}(i, j) = \text{false}$ 。

这样直接做可以在 $O(NM^3)$ 时间内算出来 $n \leq N, m \leq M$ 的答案，可以获得 12 分；进一步可以考虑优化转移，按照 i 从小到大的顺序更新后面的状态，每次形如一个区间覆盖成 true，可以做到 $O(NM^2)$ ，获得 24 分。

2.3. 算法三

考虑算法二中 DP 的过程，我们发现对于一个 i ，所有的 $\text{dp}(i, j)$ 只要被覆盖过至少一次，那么就会只剩下至多一个 j 使得 $\text{dp}(i, j) = \text{false}$ ，其余都会被覆盖成 true 。

称状态 (n, x) 表示当前还需要取 n 个石子，上一步取了 x 个。那么，对于一个 n ，只有这三种情况：

- 所有 x 都满足 (n, x) 必败。
- 所有 x 都满足 (n, x) 必胜。
- 存在唯一的 x ，使得 (n, x) 必败；其余的 x 都满足 (n, x) 必胜。

于是我们就可以直接设 f_n 表示 n 的状态，例如， $f_n = -1$ 表示所有 (n, x) 均必败， $f_n = -2$ 表示所有 (n, x) 均必胜， $1 \leq f_n \leq n$ 表示仅有 (n, f_n) 必败，其余必胜。

这样可以把 DP 的复杂度优化到单次 $O(n)$ 或 $O(n \log n)$ 。

结合算法一二，可以获得 64 分。

2.4. 算法四

考虑进一步分析转移，假如某一个 n 是必败点，即 $f_n = -1$ ，那么 $f_{n+1, \dots, n+m}$ 都不是必败的，因为它们都可以一步转移到 n 然后获胜。

这表明对于一个 m ， $[1, N]$ 中的必败点个数不超过 $\frac{N}{m}$ 。而题目只要求我们判断一个 n 是不是必败点，因此我们可以考虑预处理出所有的必败点，只有 $\sum_{m=1}^M \frac{N}{m} = O(N \log M)$ 个，然后询问的时候可以直接查表。

2.4. 算法四

进一步分析必败点的性质，考虑 $n + m + 1$ ，如果 $n + m + 1$ 不是必败点，说明存在一个 $1 \leq x \leq m$ ，使得 $(n + m + 1 - x, x)$ 是先手必败的。但我们知道对 $[n + 1, n + m]$ 中的任意一个 $n + k$ ，除了 $(n + k, k)$ 外，其余都必定是先手必胜（因为可以走到 n ），那么必须有 $m + 1 - x = x$ ，即 $x = \frac{m+1}{2}$ 。

那么对于这种情况，考虑 $n + m + 2$ ：如果此时先手选择 x ，那么

- 若 $x \geq 2$ ，后手可以选择 $m + 2 - x$ 走到必败点 n 。（注意， m 是奇数）
- 若 $x = 1$ ，注意 $\frac{m+1}{2} \neq 1$ ，因此后手仍可以选择 $\frac{m+1}{2}$ 走到必败态 $(n + \frac{m+1}{2}, \frac{m+1}{2})$ 。

因此，如果 $n + m + 1$ 不是必败点，那么 $n + m + 2$ 一定是必败点。

2.4. 算法四

那么就只需要判断 $n + m + 1$ 是不是必败点，即可求出 n 的下一个必败点。也就是说，我们需要判断 $(n + \frac{m+1}{2}, \frac{m+1}{2})$ 是不是必败态。

考虑一般的情况，我们需要判断某个 $(n + a, a)$ 判断它是不是必败态，其中 $1 \leq a \leq m$ 。那么如果第一步选择 x ，会走到 $(n + a - x, x)$ 。此时：

- 如果 $0 \leq x < a$ ，那么必须取 $x = \frac{a}{2}$ ，递归到 $(n + \frac{a}{2}, \frac{a}{2})$ 的情况。
- 如果 $a < x \leq m$ ，由于两个必败点之间的间隔至少为 $m + 1$ ，假设 n 的前一个必败点是 p ，则 $p \leq n - m - 1$ ，且 $[p, m]$ 中可能的必败态只有 $(p + b, b), 1 \leq b \leq m$ ，以及 $(n - 1, \frac{m+1}{2})$ 。
- ▶ 情况一：走到一个形如 $(p + x, x)$ 的状态，其中 $x \leq m$ 。则 $n + a - x = p + x$ ，由 $x = \frac{n-p+a}{2}$ ，可以递归下去。
- ▶ 情况二：走到 $(n - 1, \frac{m+1}{2})$ ，此时要求 $n = p + m + 2$ ，且 $(n - 1, \frac{m+1}{2})$ 已知就是一个必败态，所以如果此时 $a = \frac{m+1}{2} + 1$ ，可以直接确定 $(n + a, a)$ 必胜。

2.4. 算法四

这样可以写出一个递归的函数来判定 $(n + a, a)$ 的情况。函数的每一层至多递归出两个分枝，但可以发现这些分支都需要满足奇偶性的要求，因此平均情况下几乎不会递归很多层。实际在本地运行后可以发现跑得很快，几乎是平均 $O(1)$ 的。

这样，就可以在 $O(N \log M + t)$ 时间内解决本题，其中 $N = M = 10^6$

3. 排列计数

3. 排列计数

给定一个正整数 n ，对所有 $0 \leq x, y \leq n-1$ 求出，有多少个 $1 \sim n$ 的排列 p ，满足：

- 恰好有 x 个位置 $1 \leq i \leq n$ 使得 $p_i > i$ ；
- 恰好有 y 个位置 $1 \leq i \leq n-1$ 使得 $p_i > p_{i+1}$ 。

答案对给定的质数 M 取模。

数据范围： $1 \leq n \leq 60, 10^8 \leq M \leq 10^9$

3.1. 算法一

枚举排列可获得 15 分。

改为 $O(2^n \text{poly}(n))$ 的状压 DP 可获得 30 分。

3.2. 算法二

我们可以把排列容斥掉，即，钦定一个集合 S ，只使用 S 内部的元素，带上 $(-1)^{n-|S|}$ 的容斥系数。

我们可以从前往后，依次对每个 i 确定是否有 $i \in S$ 。

具体地，先枚举 $|S|$ ，然后我们的状态形如 $f(i, j, k)$ 表示，当前填好了前 i 个数，其中 S 在 $[1, i]$ 内包含 j 个数， p_i 是 S 中的第 k 小。这里要求 $k \leq |S|$ 。

此时， $p_i > i$ 当且仅当 $k > j$ ，由于没了排列的限制， $p_i > p_{i+1}$ 也是容易处理的。

这样只需要再记录 a, b 表示 $p_i > i, p_i > p_{i+1}$ 的个数，可以做到 $O(n^7)$ ；前缀和优化转移，即可做到 $O(n^6)$ 。可以获得 50 ~ 70 分。

3.3. 算法三

还是按照常规的思路，对 $[p_i > i]$ 和 $[p_i > p_{i+1}]$ 做容斥；只不过，对于后者，我们不再单纯地全部钦定为一种符号。具体地，我们希望容斥完之后的问题如下：

- 有若干条链，每条链的约束形如 $\langle\langle\rangle\rangle\rangle$ ，即必须单峰，峰左边有 L_i 个元素，右边有 R_i 个元素，且要求峰的值 $\leq a_i$ 。

对于这个问题，可以简单地直接用若干组合数算出答案：按照 a_i 递增的顺序填数即可。具体地，先把 a_i 排序成递增的，假设有 k 条这样的链，令 $S_i = \sum_{j < i} (L_j + R_j + 1)$ ，答案就是

$$\prod_{i=1}^k \binom{a_i - S_i}{L_i + R_i + 1} \times \binom{L_i + R_i}{L_i}$$

3.3. 算法三

现在考虑具体怎么容斥，我们记录一个 $f(i, j, k, l)$ 表示，已经决定好了前 i 个元素的容斥情况，所有的链左侧共 j 个元素，链顶有 k 个元素被钦定了 $p_i \leq i$ ，且目前有约束的所有链累计起来已经有了 l 的大小。

转移的时候，可以每次新开一条链，枚举下一条链具体长什么样。直接做的话或许会变成比较高的复杂度，需要枚举下一条链的峰的位置以及结束位置，等等。

可以分步转移，提前处理好系数，直接枚举总链长，这样可以优化到 $O(n^5)$ ，常数很小，可以获得 100 分。